UNIVERSIDADE FEDERAL DO CEARÁ

TECHNICAL REPORT

Aluno: Raimundo Rafael Vieira de Brito e Ryan Guilherme Alves de Mesquita

1. Introdução

Neste trabalho é utilizado um dataset do kaggle, ele tem dados e informações de recursos de telefones celulares e relações entre elas e suas características. Onde, é feito uma série de análises utilizando técnicas de classificação e agrupamento para avaliar a performance do algoritmo KNN e definir a melhor parametrização para a classificação do dataset.

2. Observações

O dataset de classificação foi usado o Dataset Coletado, que foi coletado na plataforma da Kaggle, ele tem dados e informações de recursos de telefones celulares e relações entre elas (por exemplo: RAM, Memória interna e preços de vendas. O objetivo não é prever o preço ideal, mas uma faixa de preço de quão alto é o preço delas.

3. Resultados e discussão

Neste relatório, foi realizada uma análise de clustering de dados usando o algoritmo KMeans. O principal objetivo foi identificar o número ideal de clusters para um conjunto de dados utilizando dois métodos: o Método do Cotovelo e o Método da Silhueta. Foi usado um dataset coletado para pré-processamento.

3.1.0 Resultados do KNN (Questão 1)

Na AV1, o melhor KNN manual foi com Chebyshev, e o GrindSearchCV apresentou o mesmo resultado, mostrando que na AV1 a métrica estava certa.

3.1.1 Determinação do Número de Clusters (Questão 2 e 3)

No gráfico do método do cotovelo indicou um ponto específico onde a inércia parava de diminuir de forma rápida.

No gráfico de silhueta foi mostrado um resultados quase iguais.

3.1.2 Distribuição dos Clusters vs Classe Alvo (Questão 4)

Por ser um dataset categórico que os clusters nem sempre seguem a divisão exata da classe alvo (blue). Isso indica que pode haver sobreposição entre as classes ou que as features utilizadas podem não separar bem os dados, mostrando dados muito próximos entre si.

4. Conclusões

Os resultados apresentados pelas implementações foram positivas,

Questão 1:

1. Introdução:

Nesta questão, o objetivo foi implementar o KNN de duas formas: O KNN manual que foi usado na AV1 e o KNN com o GrindSearchCV que utilizava as bibliotecas da scikit-learn para otimizar automaticamente os hiperparâmetros, como k e a métrica de distância. O principal objetivo da questão era comparar os resultados da AV1 com o KNN e da AV2 com o GrindSearchCV, testando diferentes métricas, como: Euclidiana, Manhattan e Chebyshev.

2. Observações:

O código estava apresentando um erro em calcular a métrica mahalanobis em uma das questões, o Mahalanobis foi testado manualmente, mas ficou dando erro por causa da limitação que a biblioteca GrindSearch possui. Portanto, optei por não utilizar essa métrica.

3. Resultados e observações:

O GridSearchCV foi utilizado para testar diferentes valores de k e métricas de distância (Euclidean, Manhattan e Chebyshev). Portanto, comparando as duas provas, na AV1, o melhor KNN manualmente implementado também apontou Chebyshev como a melhor métrica.

4. Conclusões:

O GridSearchCV foi eficiente para encontrar a melhor combinação de k e métrica, e o resultado foi consistente com a análise manual da AV1.

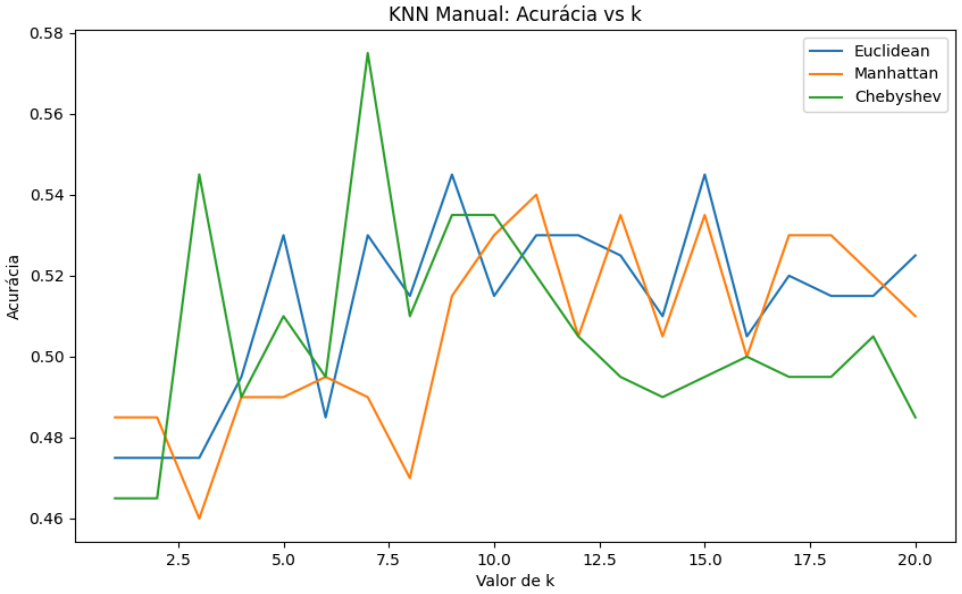
5. *Implementação:*

questao1.py M

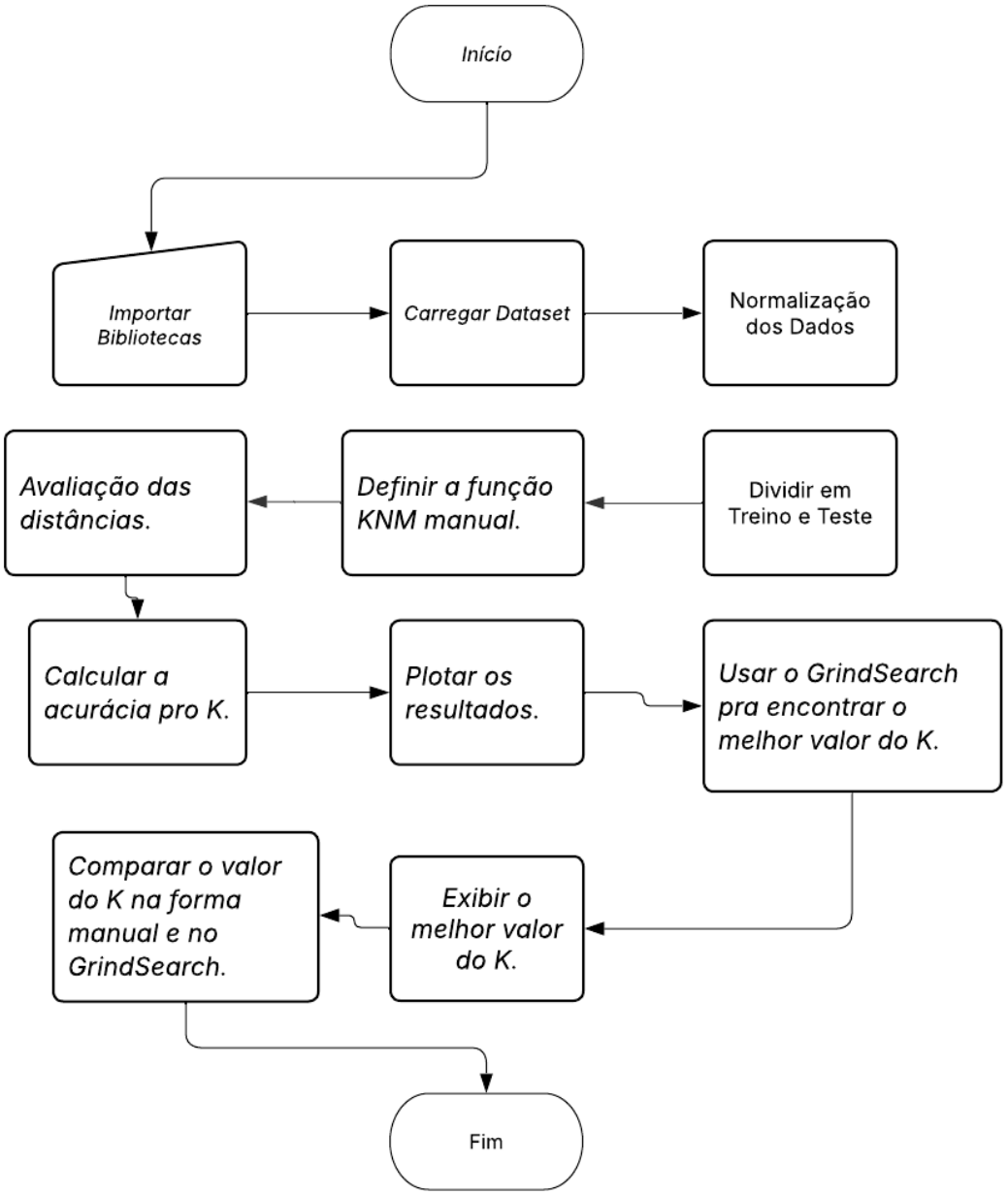
AV2 > questao1.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.distance import cityblock, chebyshev, euclidean
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from collections import Counter

dataset = pd.read_csv('Dataset_coletado.csv')

X = dataset.drop(columns=['blue'])
y = dataset['blue']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

def knn(X_train, y_train, X_test, k, distance_func):
    predictions = []
    for test_point in X_test:
        distances = [(distance_func(test_point, train_point), y_train[i]) for i, train_point in enumerate(X_train)]
        distances.sort(key=lambda x: x[0])
        k_nearest = [label for _, label in distances[:k]]
        predictions.append(Counter(k_nearest).most_common(1)[0][0])
    return predictions

distance_functions = {
    'Euclidean': euclidean,
    'Manhattan': cityblock,
    'Chebyshev': chebyshev
}

k_values = range(1, 21)
results_manual = {}
for name, func in distance_functions.items():
    accuracies = []
    for k in k_values:
        y_pred = knn(X_train, y_train.to_numpy(), X_test, k, func)
        accuracy = np.mean(y_pred == y_test.to_numpy())
        accuracies.append(accuracy)
```

```python
    results_manual[name] = accuracies

plt.figure(figsize=(10, 6))
for name, accuracies in results_manual.items():
    plt.plot(k_values, accuracies, label=name)
plt.xlabel('Valor de k')
plt.ylabel('Acurácia')
plt.title('KNN Manual: Acurácia vs k')
plt.legend()
plt.show()

param_grid = {
    'n_neighbors': list(k_values),
    'metric': ['euclidean', 'manhattan', 'chebyshev']
}

knn_sklearn = KNeighborsClassifier()
grid_search = GridSearchCV(knn_sklearn, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

best_k = grid_search.best_params_['n_neighbors']
best_metric = grid_search.best_params_['metric']
best_score = grid_search.best_score_

print(f"Melhor k encontrado pelo GridSearchCV: {best_k}")
print(f"Melhor métrica encontrada pelo GridSearchCV: {best_metric}")
print(f"Melhor acurácia no treino: {best_score:.2f}")

print("\nComparação de Resultados:")
for name in results_manual:
    best_k_manual = k_values[np.argmax(results_manual[name])]
    best_acc_manual = max(results_manual[name])
    print(f"{name}: Melhor k (Manual) = {best_k_manual}, Melhor Acurácia = {best_acc_manual:.2f}")

if best_k_manual == best_k and best_metric == name.lower():
    print("Os resultados do GridSearchCV e do KNN manual são consistentes!")
else:
    print("Houve diferenças entre os resultados do GridSearchCV e do KNN manual.")
```



6. *Fluxograma da Questão 1:*



Questão 2:

1. Introdução:

Essa questão tem uma aplicação de técnicas de clustering em um conjunto de dados. O objetivo foi determinar o número ideal de clusters a ser utilizado no algoritmo KMeans. Para isso, foi usado dois métodos:

2. Observações:

2.1 Método de Cotovelo: No gráfico de cotovelo é mostrado que com o aumento do número de clusters k a inércia vai caindo, no caso desse gráfico a melhor métrica foi k = 4 mostrando que é o número ideal para o Modelo.

2.2 Método de Silhueta: Nesse gráfico é mostrado como o coeficiente de silhueta muda de acordo com o número de k, podendo variar de -1 até 1, onde números próximos de 1 indicam que os pontos estão bem agrupados dentro do cluster. Portanto, o melhor número de cluster é k = 4.

3. Resultados e observações:

3.1 Método de Cotovelo: O gráfico de inércia mostrou que cai quando o número de clusters aumentar, a partir de k = 4 a redução da inércia foi mais suave.

3.2 Método de Silhueta: O gráfico de silhueta mostrou que o melhor valor também foi k = 4, mostrando que com clusters k = 4 o agrupamentos dos dados já é bem estruturado.

4. Conclusões:

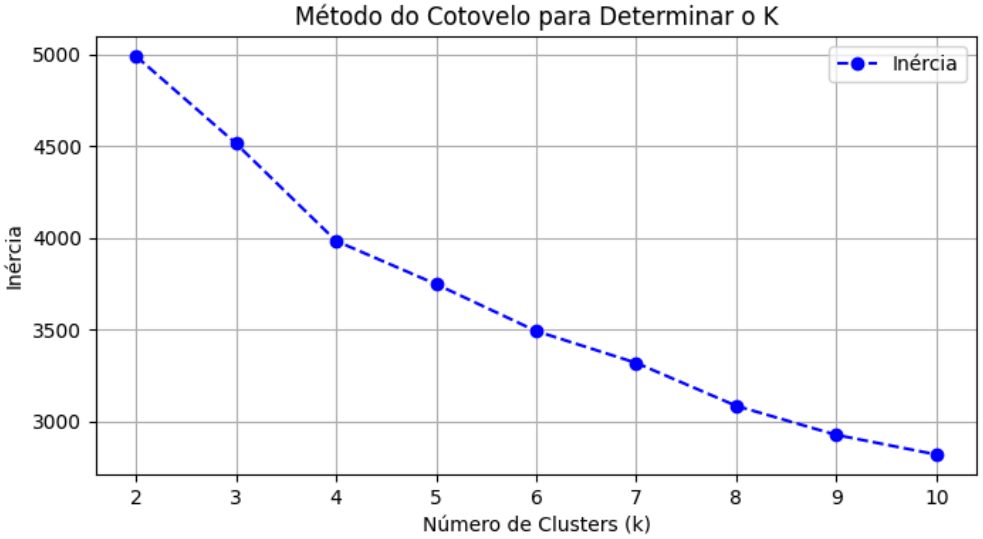
Com base nisso, dá para entender que escolher k = 4 para os dois métodos é bom para este dataset, mostrando que a inércia e a qualidade dos agrupamentos é adequada.

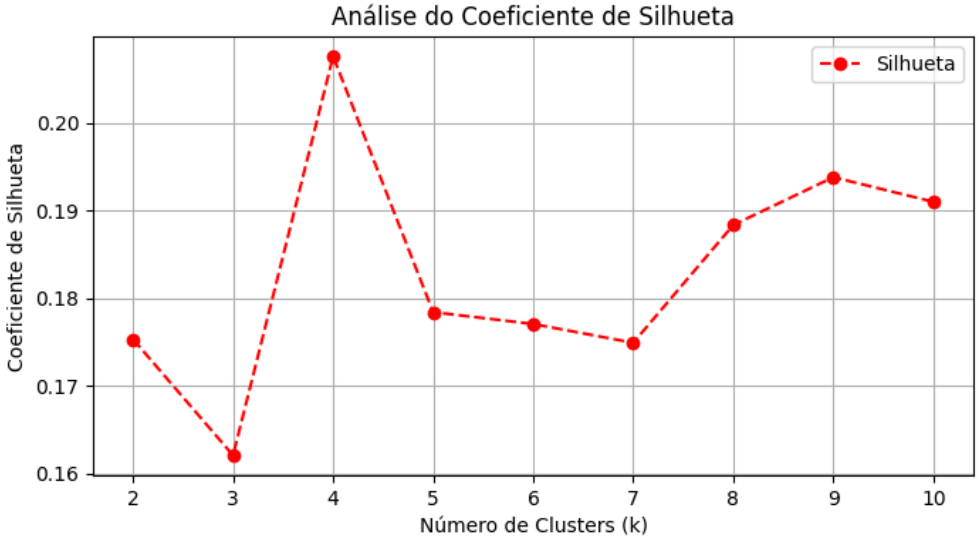
5. *Implementação:*

questao2.py M ×

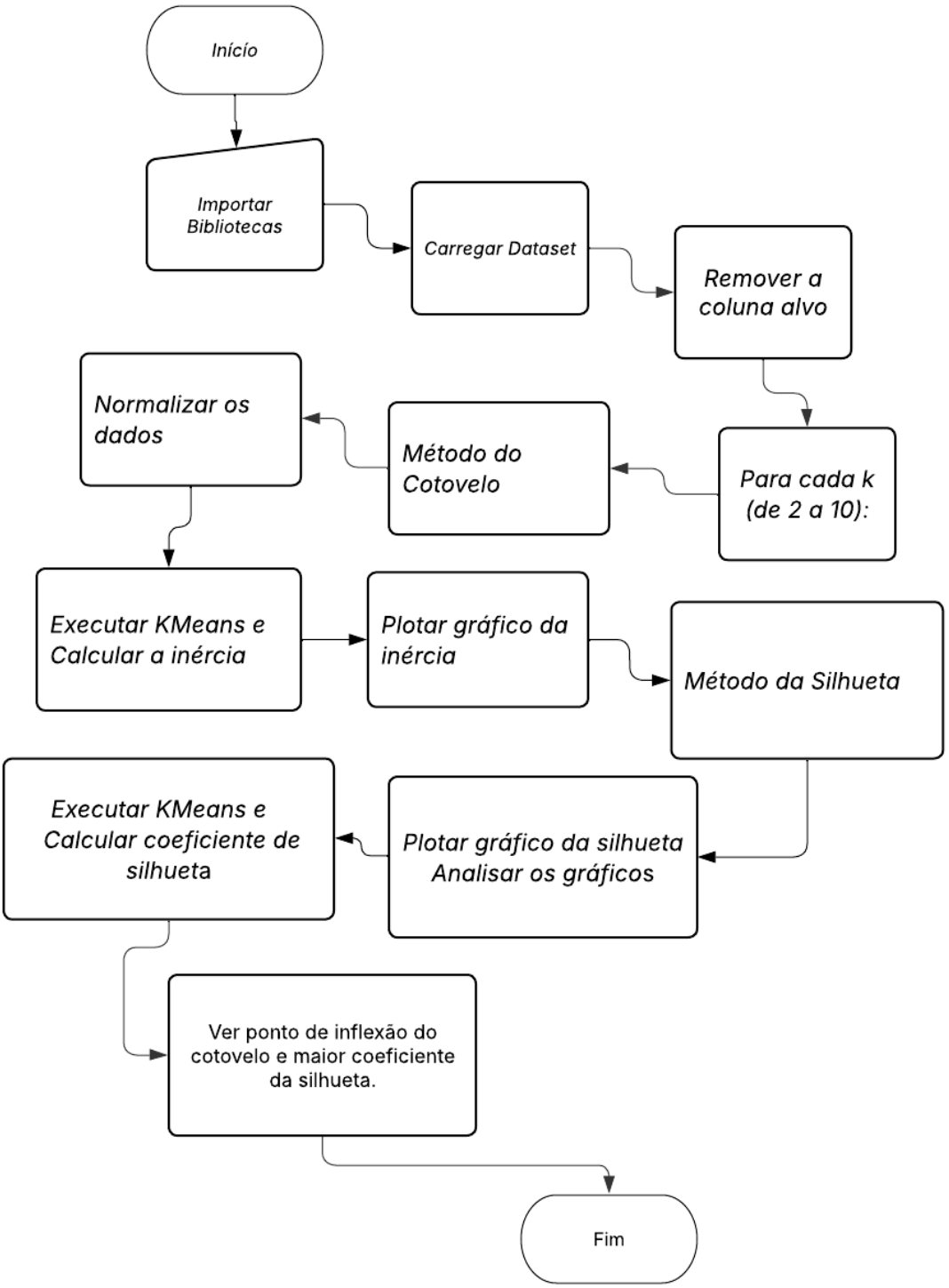
AV2 > questao2.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score


data = pd.read_csv("Dataset_coletado.csv")

if "blue" in data.columns:
    data = data.drop(columns=["blue"])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(data)

inertias = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(k_range, inertias, 'bo-', linestyle="--", label="Inércia")
plt.xlabel("Número de Clusters (k)")
plt.ylabel("Inércia")
plt.title("Método do Cotovelo para Determinar o K")
plt.legend()
plt.grid()
plt.show()

silhouette_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    silhouette_scores.append(silhouette_score(X_scaled, labels))

plt.figure(figsize=(8, 4))
plt.plot(k_range, silhouette_scores, 'ro-', linestyle="--", label="Silhueta")
plt.xlabel("Número de Clusters (k)")
plt.ylabel("Coeficiente de Silhueta")
plt.title("Análise do Coeficiente de Silhueta")
plt.legend()
plt.grid()
plt.show()
```


Método do Cotovelo para Determinar o K
Inércia
5000
4500
4000
3500
3000
Inércia
2
3
4
5
6
7
8
9
10
Número de Clusters (k)


Análise do Coeficiente de Silhueta
Silhueta
0.20
0.19
0.18
0.17
0.16
Coeficiente de Silhueta
2
3
4
5
6
7
8
9
10
Número de Clusters (k)

6. *Fluxograma da Questão 2*



Questão 3:

1. Introdução:

Esta questão detalha a implementação do processo de seleção de atributos e análise de clusters em um conjunto de dados coletados. O objetivo é encontrar os atributos mais relevantes para a separação dos dados e determinar o melhor número de clusters com base em métricas.

2. Observações:

2.1 Método do Cotovelo após Seleção de Atributos:

O gráfico mostra o Método do Cotovelo para determinar o número ideal de clusters (k) após a seleção de atributos, Mostrando que o aumento dos clusters diminui a inércia, deixando evidente que o k = 3 é ideal.

Atributos Usados: “Battery_power” e “fc”.

2.2 Método do Coeficiente de Silhueta após Seleção de Atributos:

Este gráfico avalia a qualidade dos clusters formados após a seleção de atributos, utilizando o coeficiente de silhueta. Como é mostrado no gráfico, o ponto de pico é em k= 4, indicando que é o ponto ideal de clusters.

3. Resultados e observações:

Os métodos do Cotovelo e da Silhueta foram aplicados novamente após a seleção dos atributos. Com isso, o valor ótimo de k mudou em relação à Questão 2, indicando que a remoção de atributos menos relevantes impacta diretamente na análise de agrupamento.

4. Conclusões:

A seleção de atributos com Lasso foi útil para identificar as variáveis mais importantes, o que impactou diretamente na determinação do número de clusters.

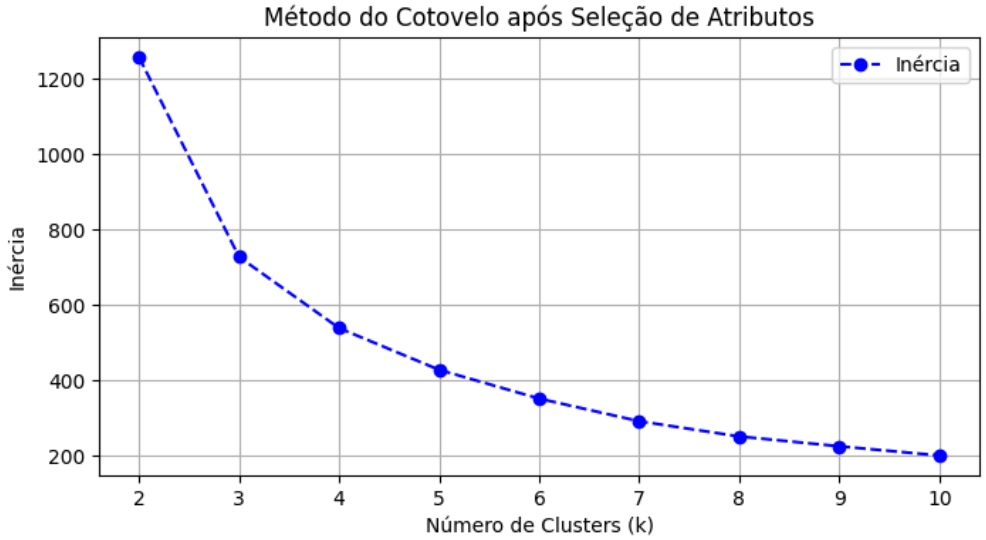
5. *Implementação:*

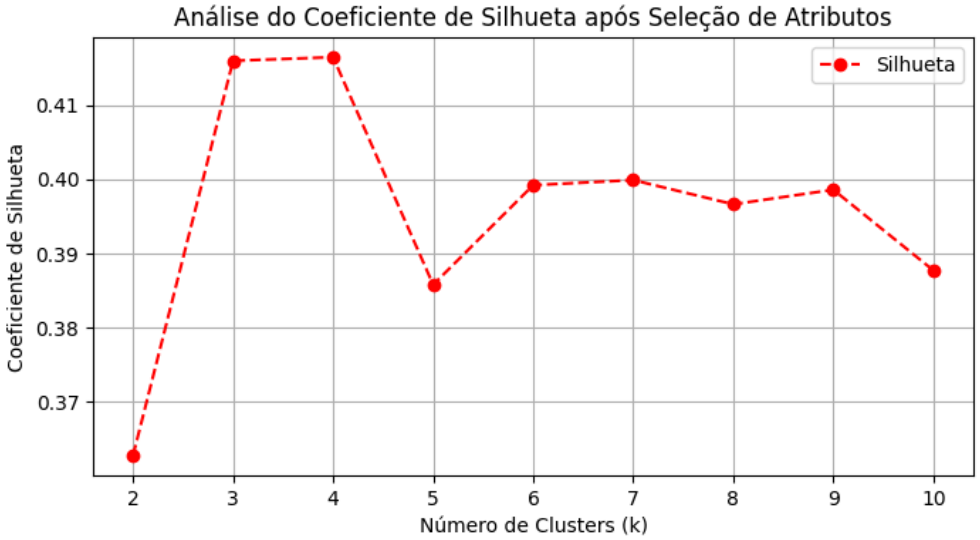
questao3.py M

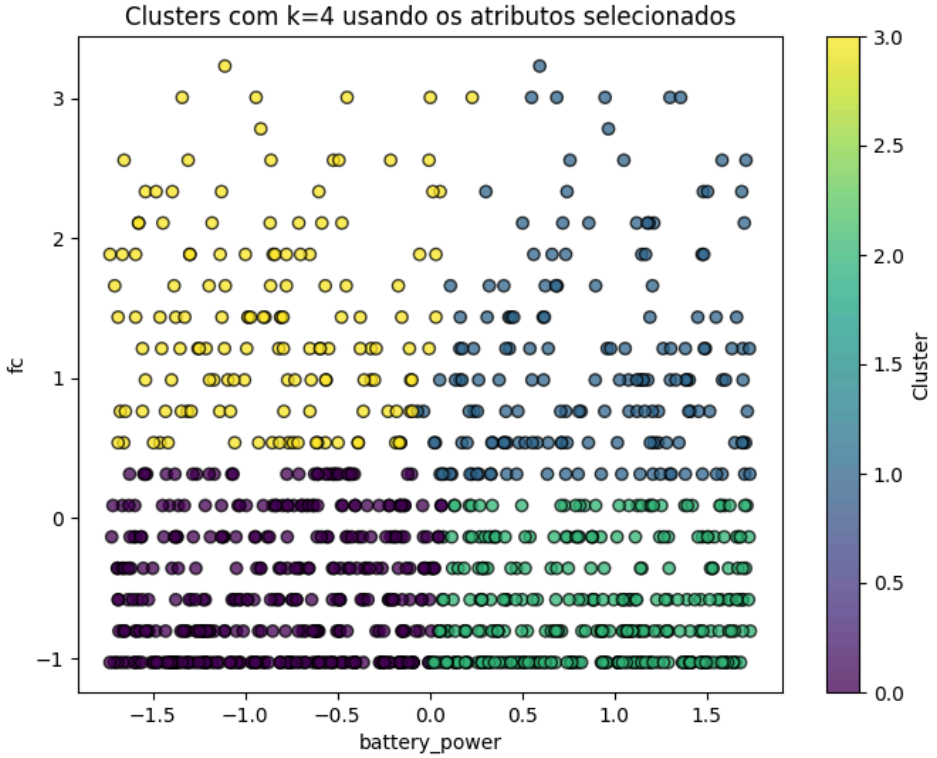
AV2 > questao3.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.linear_model import Lasso

data = pd.read_csv("Dataset_coletado.csv")


if "blue" in data.columns:
    y = data["blue"]  # Alvo
    X = data.drop(columns=["blue"])
else:
    X = data


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lasso = Lasso(alpha=0.01)
lasso.fit(X_scaled, y)

feature_importance = np.abs(lasso.coef_)
top_2_indices = np.argsort(feature_importance)[-2:]
selected_features = X.columns[top_2_indices]

print(f"Atributos mais importantes selecionados pelo Lasso: {selected_features.tolist()}")

X_selected = X_scaled[:, top_2_indices]

inertias = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_selected)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(k_range, inertias, 'bo-', linestyle="--", label="Inércia")
```

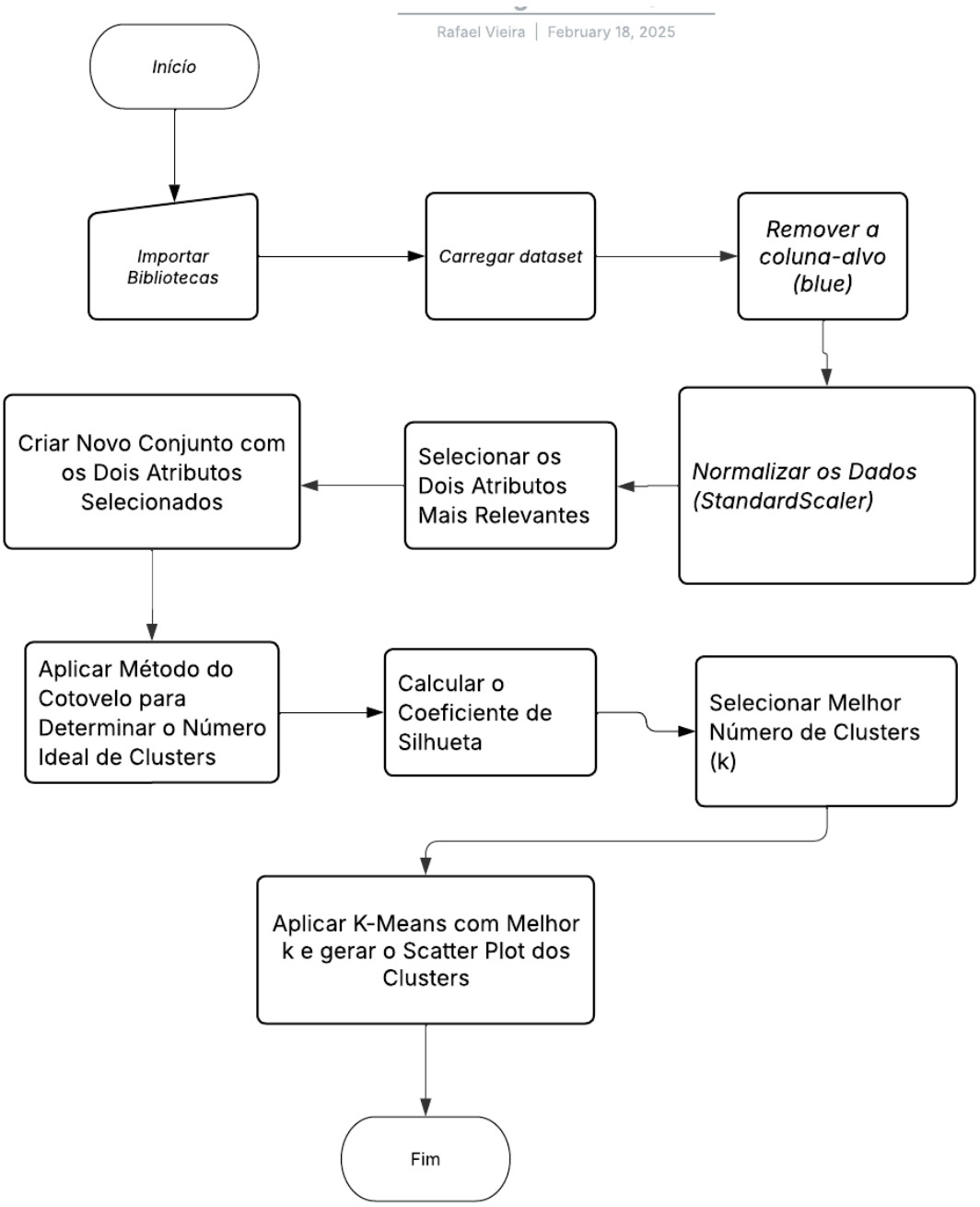
```python
plt.xlabel("Número de Clusters (k)")
plt.ylabel("Inércia")
plt.title("Método do Cotovelo após Seleção de Atributos")
plt.legend()
plt.grid()
plt.show()

silhouette_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_selected)
    silhouette_scores.append(silhouette_score(X_selected, labels))

plt.figure(figsize=(8, 4))
plt.plot(k_range, silhouette_scores, 'ro-', linestyle="--", label="Silhueta")
plt.xlabel("Número de Clusters (k)")
plt.ylabel("Coeficiente de Silhueta")
plt.title("Análise do Coeficiente de Silhueta após Seleção de Atributos")
plt.legend()
plt.grid()
plt.show()

best_k = k_range[np.argmax(silhouette_scores)]
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_selected)

plt.figure(figsize=(8, 6))
plt.scatter(X_selected[:, 0], X_selected[:, 1], c=labels, cmap="viridis", edgecolors="k", alpha=0.75)
plt.xlabel(selected_features[0])
plt.ylabel(selected_features[1])
plt.title(f"Clusters com k={best_k} usando os atributos selecionados")
plt.colorbar(label="Cluster")
plt.show()
```




Análise do Coeficiente de Silhueta após Seleção de Atributos
Silhueta
Coeficiente de Silhueta
0.41
0.40
0.39
0.38
0.37
2
3
4
5
6
7
8
9
10
Número de Clusters (k)


Clusters com k=4 usando os atributos selecionados
fc
3
2
1
0
−1
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
battery_power
Cluster
3.0
2.5
2.0
1.5
1.0
0.5
0.0

6. *Fluxograma da Questão 3*



Questão 4:

1. Introdução:

Nesta questão, o objetivo foi analisar a relação entre os clusters formados e a classe real do dataset. Para isso, foi utilizado o algoritmo K-Means com o número de clusters determinado pelo índice de Silhueta, onde os número de cluster foi obtido pelas questões 2 e 3, e então foi comparada com a variável-alvo por meio de uma tabela crosstab.

2. Observações:

A tabela crosstab revelou como os pontos do dataset foram distribuídos entre os clusters e se há alguma correspondência clara entre os clusters e as classes verdadeiras.

3. Resultados e observações:

O crosstab mostrou a distribuição dos clusters em relação às classes reais. O K-Means respondeu bem às classes. Com isso, o algoritmo conseguiu capturar padrões úteis nos dados.

4. Conclusões:

A análise mostrou que os clusters gerados pelo K-Means podem estar alinhados com as classes reais, mas a correspondência não é perfeita.

- ***Tabela gerada pelo código.***

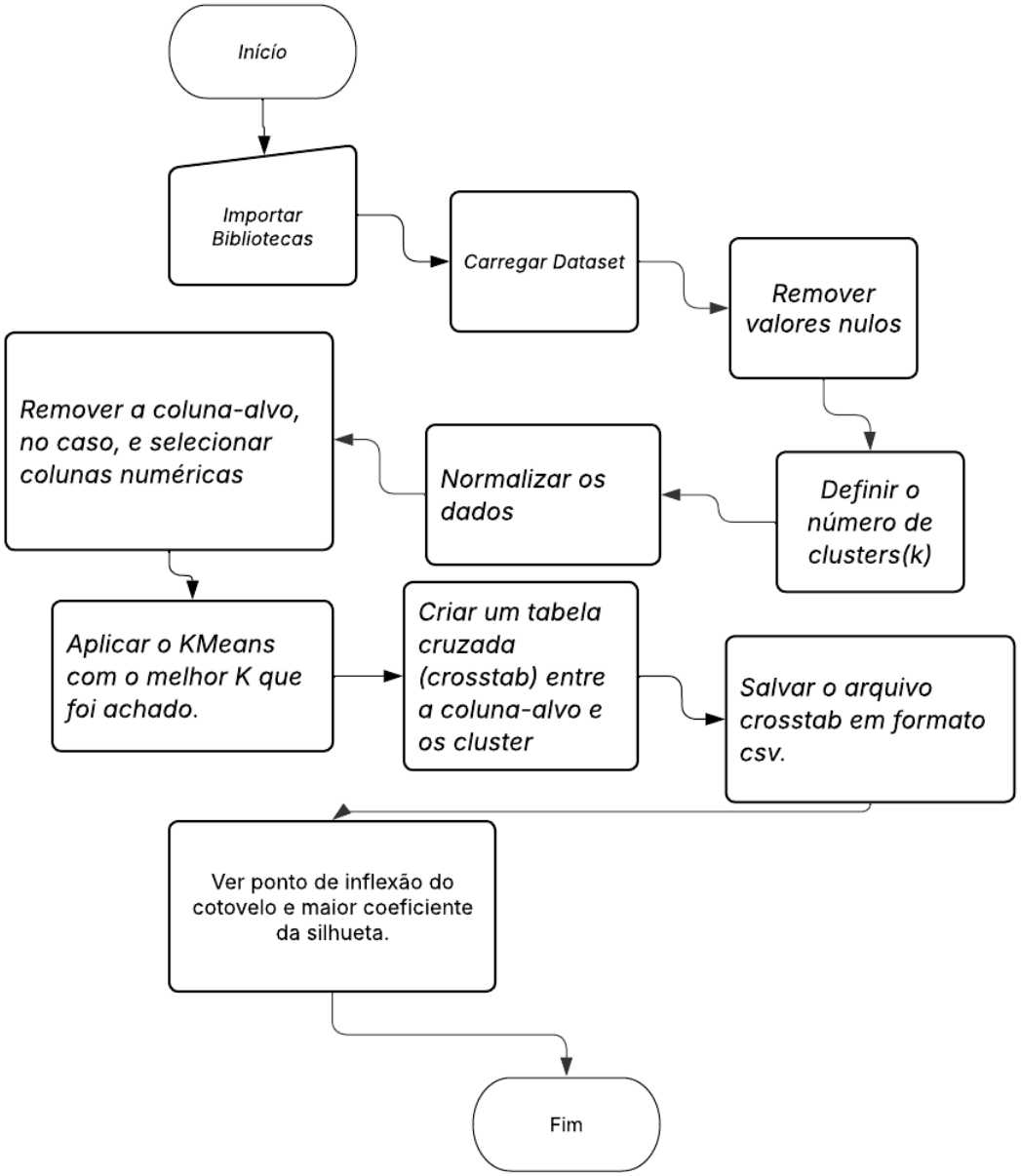
Cluster	***blue = -1.032529***	***blue = 0.968496***
0	***127***	***131***
1	***109***	***120***
2	***122***	***132***
3	***126***	***133***

Melhor valor de k pelo índice de silhueta: 4 (score = 0.2076)

5. *Implementação:*

questao4.py M ×

AV2 > questao4.py > ...

```python
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

dataset_path = "Dataset_coletado.csv"
df = pd.read_csv(dataset_path)

df = df.dropna()

coluna_alvo = "blue"
X = df.drop(columns=["blue"])
y = df["blue"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

range_k = range(2, 11)
best_k = 2
best_score = -1

for k in range_k:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)

    if score > best_score:
        best_score = score
        best_k = k

print(f"Melhor valor de k pelo índice de silhueta: {best_k} (score = {best_score:.4f})")

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

crosstab_result = pd.crosstab(df["Cluster"], df[coluna_alvo])
print("\nDistribuição de clusters por classe na coluna-alvo:")
print(crosstab_result)

crosstab_result.to_csv("crosstab_clusters.csv")
print("\nCrosstab salva em 'crosstab_clusters.csv'.")
```

6. Fluxograma da Questão 4: